Universidad Tecmilenio
Sistemas y Tecnologías de Administración del Conocimiento

Activity 3: Top-K Document Recommender with Cosine Similarity

Professor: Jesús Clemente Chio Rodríguez
Team: Ricardo Rubén Amaro Vázquez – AL07003220 · Jorge Eliud Marroquín López – AL03011384 · Juan Humberto Espinoza Arciva – AL03107716 · Edson Alfonso González Vela – AL02891583
11/05/2026

1. Description of the approach

We built a content-based recommender that, given a seed document or a short free-text query, returns the five most similar documents of an internal knowledge repository. The goal is that an employee who is reading a policy or solving a ticket immediately finds the related policies, past incidents and FAQs, instead of searching by hand.

Corpus. corpus.csv holds 40 documents (also stored as .txt files in docs_txt/) with the columns id, title, text and category. There are five categories with eight documents each: Policy/Process, Support/Incidents, HR, IT Security and FAQ. Most documents are simulated internal records (policies, meeting minutes, tickets, postmortems, FAQs), plus a few news- and post-style texts. A few short and ambiguous documents were included on purpose to observe the model's limits.

Model (Path A: TF-IDF). We chose TF-IDF for its simplicity and transparency: every score can be explained by the words two documents share. The pipeline in recommender.py is: (1) cleaning: lowercase, accent and special character removal, English stopwords; the title is joined to the text so short documents keep their topic; (2) TfidfVectorizer with unigrams and bigrams and sublinear term frequency; (3) cosine similarity between the seed vector and every document; (4) Top-5 by score, excluding the seed document itself.

Interface and code quality. main.py offers both modalities required: an interactive menu and the arguments --id and --text. The code is split into small functions with docstrings and type hints, validates the corpus (columns, duplicated IDs), handles unknown IDs and empty queries, and avoids fixed paths. run_tests.py reproduces the evidence of the tests.

Results of the tests:

Test	Input	Top-5 (score)	Coherent?
1. Policy/Process	DOC001 Travel Expense Approval Policy	DOC034 Expense reimbursement FAQ (0.23), DOC003 (0.09), DOC017 (0.09), DOC039 (0.08), DOC004 (0.08)	Yes. The top result is the practical FAQ for the same policy; the rest share generic policy wording.
2. Support/Incidents	DOC009 Ticket: VPN Will Not Connect	DOC015 (0.21), DOC035 VPN FAQ (0.20), DOC012 (0.18), DOC014 (0.10), DOC010 (0.08)	Yes. 4 of 5 are tickets and the VPN FAQ is the knowledge needed to solve the case.
3. Free text	"I received a suspicious email asking me to confirm my password..."	DOC012 Password reset (0.15), DOC026 Phishing alert (0.13), DOC035 (0.11), DOC030 MFA (0.07), DOC025 (0.04)	Partly. Phishing, MFA and password policy are right, but literal words ranked a ticket first.

An extra test with DOC016 ("Ticket: Not Working", 9 words) gave scores below 0.04 and an unrelated vacation FAQ, which confirms the weakness of TF-IDF with very short documents.

2. Areas of opportunity

- **Synonyms and meaning.** TF-IDF only matches identical words: "suspicious email" is not linked to "phishing". Using embeddings (sentence-transformers) or a hybrid TF-IDF + embeddings model would capture meaning.
- **Short and ambiguous documents.** Documents with few words produce near-zero, almost random scores. We would set a minimum score to hide low-confidence results and require minimum content in tickets.
- **Corpus bias.** All tickets share the same template ("status: closed", "priority: medium"), so the format weighs almost as much as the topic. Removing template fields before vectorizing and growing the corpus beyond 40 documents would reduce this bias.

- **Metadata and role-based filters.** Adding author, date, area, validity and confidentiality would allow filters by role or department and prevent recommending outdated policies.
- **Security.** The recommender must respect access permissions so confidential documents are never shown to unauthorized users.
- **Continuous evaluation.** Measure precision@5 against a set of relevant documents labeled by experts, collect user feedback (useful / not useful) and retrain as the repository grows.
- **Usability.** Replace the console with a simple web interface or integrate it into the ticketing tool or intranet.

3. Organizational reflection video and repository

Video link: [LINK DEL VIDEO]
Source code repository: https://github.com/SilverCoyote012/document-recommender-tfidf

The video answers why this type of recommendation reduces rework and speeds up decisions, the limitations we observed (synonyms, ambiguity, short documents, corpus bias), and what we would improve for a real-world environment.